# Scalable Secure Biometric Authentication without Auxiliary Identifiers

Alexander Bienstock[1], Daniel Escudero*[1], Antigoni Polychroniadou[1], Zhen Zeng[1], Pranav Bhat[1], Ashok Singal[1], Prashant Sharma[1], Manuela Veloso[1]

[1]JPMorganChase, New York, NY, USA.

**Abstract**

The prevalence of biometric authentication has been on the rise due to its ease of use and elimination of weak passwords. To date, most biometric authentication systems have been designed for on-device authentication of the device owner (e.g., smartphones and laptops). Recently, biometric authentication systems have started to emerge that are designed to authenticate users against cloud databases storing representations of biometrics for large numbers of users (potentially millions), such as those facilitating biometric payments. However, the use of a large cloud database introduces a significant attack vector, as a breach of the database could lead to the compromise of all enrolled users' sensitive biometric data. Indeed, all such existing systems either do not adequately protect against such a breach, or are impractical to deploy and use due to their high computational overhead. In this work, we present a new biometric authentication system that provides provable security guarantees against data breaches, while remaining scalable and performant. To do so, we marry artificial intelligence with advanced cryptographic techniques in a novel fashion, providing several optimizations along the way. Our work is the first to show that real-world scalable privacy-preserving biometric authentication without auxiliary identifiers is feasible, and we believe that it will spur widespread industrial adoption and further research in this area.

## 1 Introduction

Biometrics are becoming an increasingly popular method for user authentication, providing the means to unlock smartphones[1], access secure facilities[2], board planes[3],

*Work done while at JPMorganChase. D.E. is now at TACEO.

and more. In such systems, a user is first enrolled by providing a biometric, and thereafter that same biometric (and sometimes only that biometric, without any auxiliary information) is used to authenticate the user. Some examples of biometric modalities include face images, iris scans, fingerprints, palmprints, and more.

Biometric authentication will soon see expanded use-cases, including those where the biometric data of potentially hundreds of thousands (or millions) of users must be stored in a cloud database, rather than locally on some device. Such use-cases include biometric-based payment systems[4], which is being pushed as the payment method of the future by Amazon[5] and JPMorgan[6]; fraud prevention via ensuring biometric uniqueness across accounts in a system (e.g., banking accounts); biometric access to cloud services[7]; and more. However, the storage of large amounts of biometric data on a cloud server in these settings raises significant security and privacy concerns. Indeed, even some of the world's most mature and trusted software companies are susceptible to data breaches[8], and biometric data is particularly sensitive since it is immutable and uniquely tied to an individual. To give an illustrative example, if a person's credit card information is stolen, they can cancel the card and get a new one; if their biometric data is stolen, they cannot change something about their biology.

Once biometric data is compromised by an adversary, this adversary may try to recreate a physical copy of the biometric (for example, a fake fingerprint) in order to impersonate the victim. However, this complex physical effort may not be needed; the adversary may be able to bypass the physical biometric scanner entirely and instead infiltrate the system past the point of physical scanning, using the stolen biometric at that point. Moreover, since biometric data is soon to be used for financial transactions and other important services, the stakes are extremely high. Beyond general privacy concerns, identity theft, fraud, surveillance, and other malicious activities are all potential risks if biometric data is not properly protected. Furthermore, for companies offering biometric authentication services commercially, a data breach could lead to significant legal liability, reputational damage, and loss of customer trust[9,10]. Therefore, ensuring the security of biometric authentication systems is of paramount importance.

A pipeline for processing biometric data typically involves converting the biometric into a so-called *template*, which is a mathematical representation of the biometric that is suitable for storage and comparison. While some organizations claim that simply storing templates in the cloud database is secure enough, there are numerous works demonstrating that templates can be inverted to recover the original biometric data[11–13] (see right side of Figure 1), and thus more work is needed. Indeed, experts warn against the breach of biometric data from such systems, including that of Amazon[14].

There are some prior works that provide *provably secure* biometric authentication techniques. First, some works use advanced cryptographic primitives like *Fully-Homomorphic Encryption* and *Secure Multiparty Computation*, which allow to perform the authentication process while the biometric data remains encrypted[15–19]. However, these techniques all require some secret information to recover the result of the authentication, and this secret information must be stored somewhere. The server storing the secret information can just as easily be compromised, leading to the same issues as before. Additionally, these solutions are often impractical due to their high computational overhead[20].

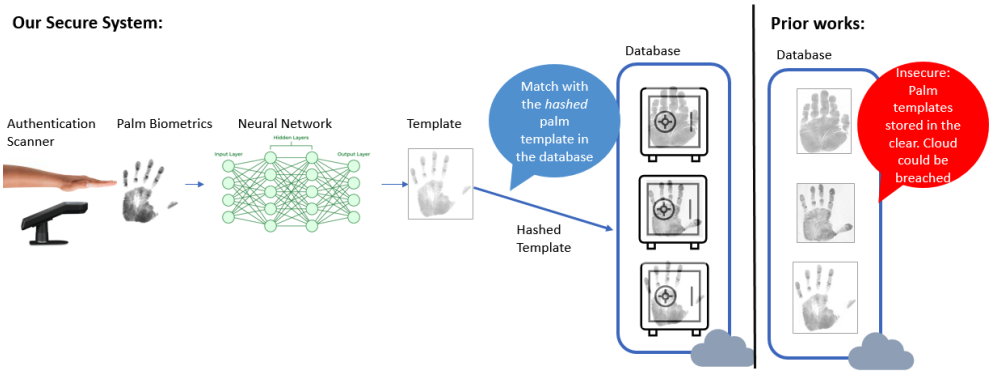



**Figure 1**: System architecture for our secure biometric authentication system. Templates computed from a neural network based on biometric scans are *cryptographically hashed* before being stored in the database, which hides all information about the biometric data. These stored hashes are matched against for authentication. Previous solutions stored templates in the clear in the database, which can be inverted to recover original biometric data in the case of a data breach.

Another line of work uses *fuzzy extractors*[21–24] to derive a strong cryptographic key from a biometric template, which is then used for authentication. Although there is no need for a secret key to be stored on the server, fuzzy extractors are only used for *one-to-one* verification of a user, in which the identity of the user is already known, explicitly through auxiliary user identifiers or implicitly through device ownership, and the purpose of the system is to verify the users's claimed identity. A richer functionality that is required in many applications is *one-to-many* identification, in which the identity of the user is not a priori known, and instead needs to be successfully determined from a large database of enrolled users, without any auxiliary identifiers. Fuzzy extractors do not natively support this richer functionality. Indeed, if a user *only* presents a biometric, and not any other identifier, then a fuzzy extractor-based system must search through the entire large database of enrolled users one-by-one to find a match, which is computationally infeasible.

In our work, we present a new biometric authentication system that addresses the above challenges. See Figure 1 for an overview of our system architecture. Our system enables for the first time accurate, secure biometric authentication using a cloud database, without the need to store sensitive information on the server. It supports real-time one-to-many identification with authentication time that scales independently of the number of enrolled users. It additionally provides for the first time an important property called *revocability*, which ensures that users can revoke their enrollment in the database in a way that guarantees their previously-enrolled biometric information cannot again be used by anyone, even if there was/is a prior/future database leak. Our solution works for all biometric modalities, however we focus on face images in our experiments due to the availability of large-scale public datasets (and lack thereof for other modalities). We leverage advanced cryptographic techniques to ensure the privacy and security of biometric data throughout the lifetime of the system, while also

ensuring practicality for real-world use cases. The properties achieved by our system in comparison to previous practical systems are listed in Table 1.

| | Encryption in Storage/Computation | Authentication Time | Revocability |
|---|---|---|---|
| Existing Practical Systems | No | $O(N)$ | No |
| Our Secure System | **Yes** | $\mathbf{O(1)}$ | **Yes** |

**Table 1**: Comparison of our secure biometric authentication system with existing practical systems. $N$ is the number of enrolled users. Our system encrypts biometric data at all times, has authentication time independent of number of enrollees, and provides revocability (defined above), while existing practical systems do not have any of these properties.

## 2 System Architecture and Threat Model

We now overview the architecture of our system (depicted in Figure 1), along with the threat model. A more formal model is presented in Appendix A. Our system consists of two types of devices: scanners that capture biometric data from users, and a remote database that receives information extracted from the biometric from the scanner, stores it and determines if authentication was successful.

During the enrollment phase, a user registers their biometric data in the system, in order to authenticate at some later point, using the same biometric. The system stores some derivation of their biometric data associated with their identity in the database, alongside the (processed) data of all other users of the system. Then, during the authentication phase, a user attempts to authenticate with the system using their biometric data. The system checks whether the biometric data corresponds to a user in the database, and if so, returns the user's identity.

From a functionality standpoint, we naturally only want enrolled users to be able to successfully authenticate as themselves. For security, we assume that the scanners are trusted devices that securely capture biometric data and send extracted information to the database over an encrypted channel. In practice, the scanners immediately delete any data derived from the biometrics and would be fortified with anti-tampering measures[25,26], liveness detection[26], and other security features to prevent adversarial manipulation. On the other hand, the remote database may be compromised, and our goal is to ensure that even if an adversary gains access to the database, they cannot learn any sensitive information about enrolled users' biometrics.

***Entropy Assumption.***

A primary assumption underlying secure biometric authentication systems is that biometric data is sufficiently random; i.e., has *high entropy*. Indeed, if biometric data was not very random, then an adversary could easily learn the distribution of biometric data, and simply sample likely biometric values until they find one that

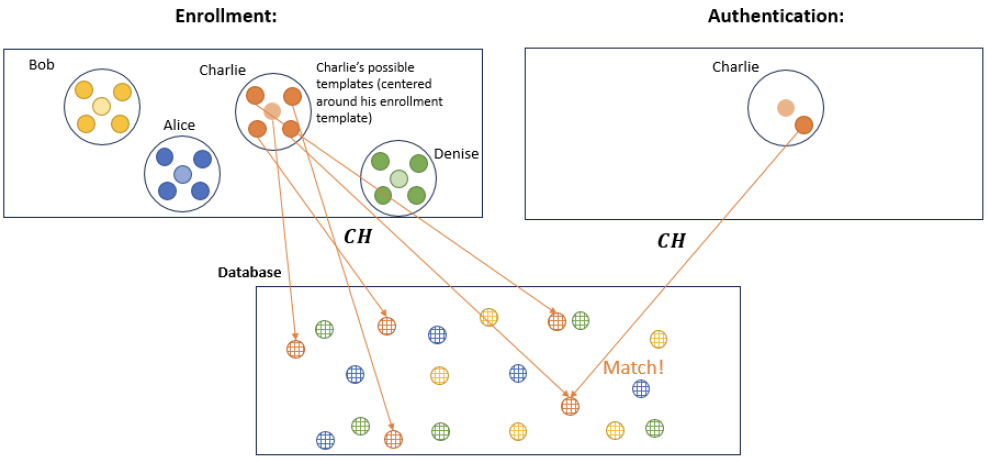



**Figure 2**: Pictorial, simplified overview of our secure biometric authentication protocol. During enrollment, the system cryptographically hashes all possible templates around the user's enrollment template (centered and faded) using CH and stores these hash values in the database, paired with the user's identity (represented by color). During authentication, the system hashes the authentication template and checks if the resulting hash value matches any of the stored hash values for any user. Note that all hashed values stored in the database are randomly scattered throughout the space, depicting the security of the system.

successfully authenticates as some enrolled user. As explained in the Introduction, the adversary need not physically recreate the biometric, but could instead bypass the physical biometric scanner and infiltrate the system past that point, using the sampled biometric. Fortunately, prior works have provided empirical evidence that biometric data does indeed have high entropy[23,27], and in Section 5.2 we provide further empirical validation of this assumption for our specific system instantiation. In our system, we will in fact rely on this assumption of intrinsic randomness of biometrics to actually protect information about the biometric itself (see the next section and Appendix A).[1]

# 3 Protocol Overview

Here, we provide a general overview of our system. We start with the baseline one-to-one sample-then-lock fuzzy extractor of Canetti *et al.*[22] (improved upon by Shukla *et al.*[23]) and with novel techniques, extend it to the one-to-many authentication setting we aim to achieve in this work. In Appendix B, we provide a formal specification of our protocol and in Appendix C we provide a security proof for our protocol.

Our system critically relies on a base templatization model that extracts a high-dimensional, real-valued *template* (or embedding) from a given biometric scan. This model ensures that templates extracted from biometrics of the same person are *close*,

[1] Indeed, we can tune the amount of entropy captured by our system, and thus the security of our system.

while templates extracted from different people are *far*, both in terms of *euclidean distance* (which is closely related to *cosine similarity*).

In addition, we require a *cryptographic hash function*, $\mathsf{CH}$, approved by the National Institutes of Standards and Technology, such as SHA-3[28], that maps bit strings to bit strings. The main property of a cryptographic hash function that we utilize is that if the input is sufficiently random (i.e., has high entropy), then the output will look random, and it will be computationally infeasible to recover any information about the input from the output. Cryptographic hash functions are *deterministic*, meaning that the same input will always yield the same output.

With these two building blocks, we can provide the simplified intuition of our system, depicted pictorially in Figure 2. The templatization model gives the property that all possible authentication templates for a given user will be closely centered around their enrollment template, and far away from the possible templates of all other users. Thus, we can cryptographically hash all possible templates around each users' enrollment template, and store these hash values, paired with the user's identity, in the database.[2] Then, during authentication, we can hash the authentication template and check if the resulting hash value matches any of the stored hash values for any user. Correctness is achieved, since from above, this authentication hash value will match *only* one of the hashes stored in the database that we computed during enrollment for that user, and not any other user. Moreover, we achieve security assuming that biometric data has high entropy, since we only store outputs of the cryptographic hash function in the database.

Below, we introduce the system in more detail. To bridge the gap between the real-valued templates and the discrete inputs of cryptographic hash functions, we also utilize a *locality-sensitive hash* (LSH). An LSH maps templates into bit strings, such that the output bit strings of two close templates are close in hamming distance (i.e., the number of bits where the strings differ), and two far templates result in bit strings that are far in hamming distance.

***Global Setup.***

The system starts with some global setup. We denote the output length of the LSH as $n$. Additionally, let $0 < k < n$ and $m > 0$ be some other system parameters. The system will first generate $m$ length-$k$ subsets of $[n] \coloneqq \{1, 2, \ldots, n\}$, denoted $\mu_1, \mu_2, \ldots, \mu_m$. An example for how to generate such subsets is just uniformly random and independently; see other methods in Section 4.3. We will also use $t \coloneqq n - k$ to denote the number of elements of $[n]$ that we exclude from each subset $\mu_i$.

***Enrollment Phase.***

We now provide a more detailed description of the enrollment phase, demonstrated in the top of Figure 3. After capturing the user's biometric data, the system runs it through the templatization model to obtain a template. Next, this template is run through the LSH to obtain a *bit string* of length-$n$, $v \coloneqq (v_1, \ldots, v_n)$, where each $v_i \in \{0, 1\}$. With $v$ in hand, the system next computes all length-$k$ substrings of $v$ corresponding to $\mu_1, \mu_2, \ldots, \mu_m$ generated during the global setup phase. We denote

[2] Our final system performs this step in a more nuanced, efficient way.

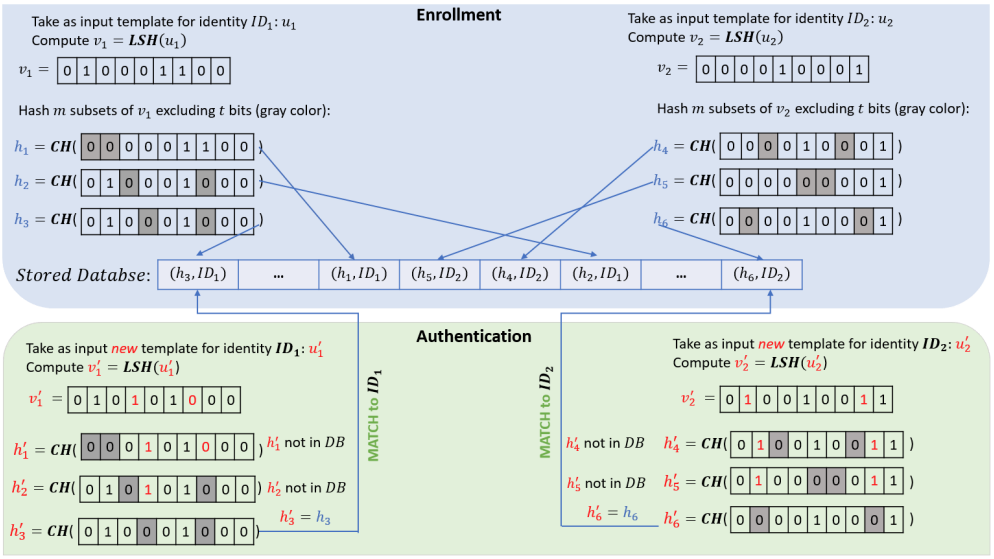



**Figure 3**: More detailed overview of our secure biometric authentication protocol, demonstrating the cryptographically hashed values of bit substrings that are stored in the database during enrollment (top) and against which authentication is performed (bottom). In this example, we have $m = 3$, $n = 9$, and $k = 7$ (thus $t = 2$). Since LSH output strings of the same user are close and LSH output strings of different users are far, the cryptographically hashed values of substrings will only match for same users, ensuring correctness.

these resulting substrings as $w_1, w_2, \ldots, w_m$. Finally, the system cryptographically hashes each $w_i$ using $\mathsf{CH}$ to obtain a hash value $h_i := \mathsf{CH}(w_i)$ and stores these hash values $h_1, \ldots, h_m$ in a database, $\mathsf{DB}$, under the user's identity, $\mathsf{ID}$. This database is a hash table, where the keys are the hash values and the values are the user identities; i.e. $(k, v) := (h, \mathsf{ID})$. Such a hash table allows for constant-time lookups of keys (i.e., hash values).

### *Authentication Phase.*

Authentication follows a similar pattern to enrollment, presented in the bottom of Figure 3. Indeed, authentication starts in the same way, by capturing the biometric, running it through the templatization model and LSH, obtaining substrings $w_1, w_2, \ldots, w_m$ according to global subsets $\mu_1, \mu_2, \ldots, \mu_m$, and hashing each substring to obtain hash values $h_1, h_2, \ldots, h_m$. Then, instead of storing the hashes as in enrollment, the system performs a lookup (in constant time) on each obtained hash $h_i$, within the database. If the system finds a number of matching hashes for some identity $\mathsf{ID}$ that is greater than some threshold $\tau$, it returns $\mathsf{ID}$; otherwise, it returns $\perp$ to indicate that authentication has failed.

***Correctness.***

For the same person, the templatization model and LSH guarantee that different biometric scans result in bit strings $v$ and $v'$ that are close to each other, with high probability. As a result, Canetti *et al.*[22] observed that some number of the $m$ substrings $w$ of $v$ will match the corresponding substrings $w'$ of $v'$, with high probability. Thus, with high probability, there will be enough hashed values during authentication time that match with the given user's enrolled hashes. On the other hand, for different people, since the templatization model and LSH guarantee that their corresponding biometric scans will result in bit strings $v$ and $v'$ that are far apart, very few (if any) of the substrings of $v$ will exactly match any of the $m$ substrings of $v'$, with high probability. Therefore, with high probability, not enough hashed value matches will be found during authentication time, and thus the system does not allow incorrect authentication. See Section 5.2 for experimental verification of these phenomena.

***Security.***

Security comes from our assumption that biometric data has *high entropy*. Indeed, if this is the case, and the parameters of the templatization model, LSH, and substring length $k$ are chosen appropriately, then each resulting bit string $w_i$ will also have high entropy. Therefore, the cryptographic hash function will output *random-looking* hash values $h_i$, that cannot be used to recover information about the original hashed substrings $v_i$ (nor the original biometric data or template). In Section 5.2 we empirically validate these assumptions by employing standard entropy estimation[23,27] to show that the hashed substrings indeed have high entropy.

# 4 Concrete System Instantiation

In this section, we give concrete instantiations of the abstracted components of our high-level protocol from the previous section.

## 4.1 Templatization Model

For our templatization model, we use Arcface[29] with 1024-dimensional templates. Arcface adds additive angular margin loss to the loss function to maximize class separability, while pushing members of the same class closer together. This increases accuracy on facial recognition tasks and intuitively also increases the amount of entropy that our system can capture. Indeed, if templates for different identities are more separated and those for the same identity are closer together in euclidean space, this will translate to the corresponding LSH outputs having higher hamming distance and lower hamming distance for different and same identities, respectively. Thus, longer substrings of these outputs will be much less likely to collide between different identities, while for the same identity, the likelihood of collision will remain high. Therefore, we can sample longer substrings and thus capture more entropy in each of these substrings, while maintaining low error rates.

### 4.2 Locality-Sensitive Hash Function

For the LSH, we use the state-of-the-art OrthoHash[30] with 4096-bit outputs. Orthohash is based on *deep hashing*—i.e., deep learning techniques, followed by some binarization process—instead of traditional statistical techniques like random projections. The objective function used in OrthoHash simultaneously results in discriminative outputs, i.e., intra-class distances are small while inter-class distances are large, and also minimizes the quantization error from binarizing the continuous outputs.

### 4.3 Subset Generation

For the generation of subsets, we use $\zeta$-*sampling* from Shukla *et al.*[23]. Instead of using the naive approach of sampling each bit uniformly at random for each substring, $\zeta$-sampling weights the bits according to some measure. While, Shukla *et al.* examine a few different such measures, we focus on a mutual-information based approach, which intuitively measures how much each bit in the LSH output reduces the uncertainty about the identity of the biometric:

$$\mathsf{mi}_i \coloneqq I(\mathsf{bit}_i; \mathsf{ID}) = H(\mathsf{ID}) - H(\mathsf{ID}|\mathsf{bit}_i),$$

where $H$ is the entropy function, $\mathsf{bit}_i$ is a random variable representing the value of bit $i$, and $\mathsf{ID}$ is a random variable representing the identity of the biometric. Then, the sampling weight $\mathsf{wt}_i$ of each bit $\mathsf{bit}_i$ is proportional to this mutual information value, potentially further weighted by some value $\zeta$:

$$\mathsf{wt}_i \coloneqq \frac{\mathsf{mi}_i^\zeta}{\sum_j \mathsf{mi}_j^\zeta}.$$

This mutual-information based approach simultaneously increases accuracy (as bits that are more informative about the identity are more likely to be sampled) and also increases the entropy captured in each substring (as bits that are more discriminative are more likely to be sampled, and thus the length of substrings can be longer).

## 5 Evaluation

Now we provide empirical evaluation of our system, showing that it is efficient, accurate, and secure.

### 5.1 Evaluation Dataset

Due to the lack of available large-scale public datasets of high-quality face images with multiple images per identity, we use synthetic face images generated by the GAN-Control library[31]. This library allows us to generate multiple realistic images of the same synthetic identity with control over facial expressions, lighting conditions, head poses, etc. We generate two images each for approximately 100,000 synthetic identities. See Figure 4 for examples of synthetic images used in our experiments.


**Figure 4**: Comparison of random expressions (top) and fixed expressions (bottom) for 5 different identities. Each ID shows two images side by side.

## 5.2 Experimental Results

We now present the experimental results for our system, first explaining the insecure baseline we use for comparison, then describing the different types of experiments we run, and finally discussing the results of these experiments, including accuracy, computational efficiency, and entropy estimation.

***Insecure Baseline.***

This insecure baseline simply stores the output of the ArcFace templatization model on the enrollment image of every identity, then during authentication, computes the euclidean distance between the template of the authentication image and the stored template for each enrolled identity, and authenticates if the distance is below some threshold.

***Experiment Types.***

We run experiments on two sets of data: (i) face images with different random expressions for each identity used during enrollment and authentication, and (ii) face images with two similar expressions for each identity used during enrollment and authentication, with the two expression styles fixed across identities. See Figure 4 for examples of these two sets of data. While (i) is a more challenging and realistic setting, publicly available templatization models are vastly inferior to (closed-source) industrial solutions[32]. Moreover, as discussed earlier, the better the model is at separating the distance between same biometrics and different biometrics, the more entropy our system can capture, and thus the more secure it is. Therefore, we use (ii) to come closer to matching the accuracy of industry models, and thus to show that our system can achieve very low error rates while also capturing a meaningful amount of entropy, which is crucial for security. We still use (i) to empirically estimate the additional error that our secure system incurs compared to the insecure baseline of ArcFace, and to show that even in this more challenging setting, the error increase is not too large.

In both settings, we evaluate the standard measures of False Negative Rate (FNR) and False Positive Rate (FPR). FNR is the rate at which enrolled identities do not

properly authenticate as themselves; FPR is the rate at which non-enrolled identities improperly authenticate as someone in the enrolled database. We will also refer to generic *accuracy*, for which we use the average of FNR and FPR as the *error rate*.

***Experiments with Random Expressions.***

We start by running experiments on synthetic faces generated with random expressions (top of Figure 4). As Extended Data Figure 1 shows, the parameters of our system that yield the lowest error results in a $\lesssim 1.5\times$ increase in the error rate compared to the baseline insecure version; this means that if the baseline insecure system has 99.9% accuracy, our system will have $\approx 99.85\%$ accuracy. Unfortunately, as mentioned earlier, the accuracy of even the baseline insecure system based on arcface for faces with random expressions is quite low. Moreover, as a side effect, the difference between distances of same and different identities is not large, and thus the length of substrings and entropy captured at this accuracy is quite low (as explained above). This low accuracy of the baseline system is due to a number of reasons, most notably due to the lack of large high-quality public datasets of face images needed for successful training; it is known that industry models are able to achieve much better accuracy as high as $\approx 99.95\%$ for databases of size 12 million[32].

***Remedy for Low Captured Entropy.***

We propose a remedy to the above low entropy issue. Indeed, although the length of the substrings and thus the entropy captured is low, if we sample a random cryptographic key and use it as input to a (also deterministic) Pseudo-Random Function (PRF)[33] to generate the outputs instead of a cryptographic hash function, then the outputs will be computationally indistinguishable from random, as long as the key remains secret. One way to keep the key secret is to store it in a Trusted Execution Environment[34], which is a secure environment isolated that is inaccessible from the rest of a server's operating system, and have all PRF computations happen inside this secure environment. Thus, our security objective can still be achieved, albeit under a different assumption that the PRF key cannot be retrieved from the TEE.

***Experiments with Similar Expressions.***

To come closer to matching the accuracy of industry models, we run experiments on synthetic faces with fixed, similar expressions (bottom of Figure 4). As shown in Figure 5 (left), our secure system can achieve very low error rates in this setting, across database sizes. Indeed, even for substring lengths up to 110 bits, we maintain very low and performant error rates. As we increase the substring length, increasing the security of the system, we see a gradual increase in *only* false negatives, but the system remains largely performant.

***Computational Efficiency.***

We now discuss computational efficiency of our system. First, for storage per identity, our sytem involves storing $m$ key-value pairs, where the key is a hash value and the value is the identity of the user. The hash value length we use is the standard 256 bits and we use 32-bit integers to represent identities. Thus, the total storage per identity

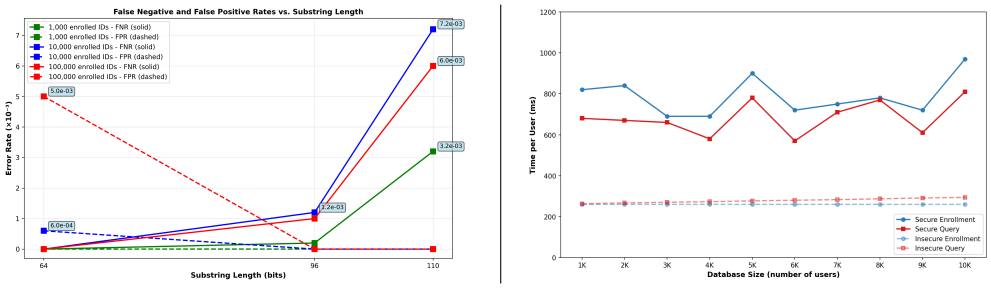



**Figure 5**: **Left**. Error rates for databases of size 1,000, 10,000, and 100,000 enrolled IDs. FNR: False negative rate; FPR: False Positive Rate. False negatives are calculated over first 1000 enrolled identities. False positives are calculated over 1000 non-enrolled identities. Runs of our secure system are averaged over 5 trials. **Right**. Average time per user for enrollment and authentication queries; both secure (solid) and insecure (dashed, faded).

is $m \cdot (256 + 32)$ bits. We use $m = 250$K for the experiments in Figure 5 (left), which results in a storage of $\approx 9$ MB per identity, which is quite reasonable for modern servers. However, due to memory constraints, we recommend sharding the database so that each shard contains the (hash value, identity) pairs for at most 10K identities. Authentication queries would be processed by each shard in parallel, whose results would be processed together by an authentication server.

With this in mind, in Figure 5 (right) we present the average time per user for both enrollment and authentication queries, for database sizes up to 10K. Both enrollment and queries take under 1 second for all database sizes and queries take as little as 680 milliseconds for database size of 1K. We note that even though in theory, enrollment and query time should be fixed across database sizes, since we are only performing constant-time hash table operations, both actually slightly increase with larger databases. This is because as the enrollment database grows, the hash table used for storing and looking up biometric templates becomes larger, leading to increased CPU cache misses during dictionary lookups, and therefore slower main memory fetches. Compared to the insecure baseline, our system maintains competitive enrollment and query times.

***Entropy Estimation.***

Measuring the true entropy of samples from a distribution requires an exponentially large number of samples[35,36]. However, there are established methods for estimating the min-entropy of biometrics in the literature. To estimate the min-entropy of the substrings that we hash in our system, we use the standard method of Shukla *et al.*[23], which is a modification of Daugman's test[27]. This estimation involves computations on the mean, $\mu_{\mathsf{Unlike}}$, and standard deviation, $\sigma_{\mathsf{Unlike}}$, of pairwise Hamming distances between the substrings of different identities in the dataset:

$$e = \frac{-\mu_{\mathsf{Unlike}}(1-\mu_{\mathsf{Unlike}})\log\max\{\mu_{\mathsf{Unlike}}, 1-\mu_{\mathsf{Unlike}}\}}{\sigma^2_{\mathsf{Unlike}}}.$$

In Extended Data Table 1, we show that the minimum (most important for security guarantees) of estimates of min-entropy for all substrings is quite high, reaching 74.3 bits of entropy for substring lengths of 110 bits, which is a very meaningful level of security. We also provide the mean and maximum estimates across substrings for reference.

## Disclaimer



## References

[1] Abuhamad, M., Abusnaina, A., Nyang, D. & Mohaisen, D. Sensor-based continuous authentication of smartphones' users using behavioral biometrics: A contemporary survey (2020).

[2] Poh, N. *et al.* Benchmarking quality-dependent and cost-sensitive score-level multimodal biometric fusion algorithms (2009).

[3] Khan, N. & Efthymiou, M. The use of biometric technology at airports: The case of customs and border protection (cbp) (2021).

[4] Patra, G. K., Rajaram, S. K., Boddapati, V. N., Kuraku, C. & Gollangi, H. K. Advancing digital payment systems: Combining ai, big data, and biometric authentication for enhanced security (2022).

[5] Amazon. Amazon one. https://amazonone.aws.com/help#retail. Amazon.

[6] J.P. Morgan. Biometric payments for faster, safer checkouts. https://www.jpmorgan.com/payments/solutions/commerce/omnichannel/biometric-payments. J.P. Morgan.

[7] Talreja, V., Ferrett, T., Valenti, M. C. & Ross, A. Biometrics-as-a-service: A framework to promote innovative biometric recognition in the cloud (2018).

[8] Moore, T. On the harms arising from the equifax data breach of 2017 (2017).

[9] Ponemon Institute & IBM. Cost of a data breach report 2024 (2024). URL https://www.ibm.com/reports/data-breach.

[10] Federal Trade Commission. Policy statement of the Federal Trade Commission on biometric information and section 5 of the Federal Trade Commission Act (2023). URL https://www.ftc.gov/legal-library/browse/policy-statements/policy-statement-federal-trade-commission-biometric-information-section-5-federal-trade-commission.

[11] Gomez-Barrero, M. & Galbally, J. Reversing the irreversible: A survey on inverse biometrics (2020).

[12] Shahreza, H. O. & Marcel, S. Template inversion attack against face recognition systems using 3D face reconstruction (2023).

[13] Mai, G., Cao, K., Yuen, P. C. & Jain, A. K. On the reconstruction of face images from deep face templates (2019).

[14] DeVon, C. Amazon biometric payments privacy concerns. https://www.cnbc.com/2023/08/26/amazon-biometric-payments-privacy-concerns.html (2023). CNBC.

[15] Boddeti, V. N. Secure face matching using fully homomorphic encryption (2018).

[16] Engelsma, J. J., Jain, A. K. & Boddeti, V. N. HERS: Homomorphically encrypted representation search (2022).

[17] Choi, H., Choi, J., Yoon, M., Cheon, J. H. & Cho, Y. Blind-match: Efficient homomorphic encryption-based 1:N matching for privacy-preserving biometric identification (2024).

[18] Benhamouda, F. *et al.* Huang, C.-Y., Chen, J.-C., Shieh, S.-P., Lie, D. & Cortier, V. (eds) *Encrypted matrix-vector products from secret dual codes.* (eds Huang, C.-Y., Chen, J.-C., Shieh, S.-P., Lie, D. & Cortier, V.) *ACM CCS 2025*, 394–408 (ACM Press, 2025).

[19] Bloemen, R., Kales, D., Sippl, P. & Walch, R. Large-scale MPC: Scaling private iris code uniqueness checks to millions of users. Cryptology ePrint Archive, Report 2024/705 (2024). URL https://eprint.iacr.org/2024/705.

[20] Yang, W., Wang, S., Cui, H., Tang, Z. & Li, Y. A review of homomorphic encryption for privacy-preserving biometrics. *Sensors* **23**, 3566 (2023).

[21] Dodis, Y., Ostrovsky, R., Reyzin, L. & Smith, A. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data (2008). URL https://doi.org/10.1137/060651380. https://doi.org/10.1137/060651380.

[22] Canetti, R., Fuller, B., Paneth, O., Reyzin, L. & Smith, A. D. Reusable fuzzy extractors for low-entropy distributions. *Journal of Cryptology* **34**, 2 (2021).

[23] Shukla, A. *et al.* Huang, C.-Y., Chen, J.-C., Shieh, S.-P., Lie, D. & Cortier, V. (eds) *Fuzzy extractors are practical: Cryptographic strength key derivation from the iris.* (eds Huang, C.-Y., Chen, J.-C., Shieh, S.-P., Lie, D. & Cortier, V.) *ACM CCS 2025*, 3605–3619 (ACM Press, 2025).

[24] Uzun, E., Yagemann, C., Chung, S. P., Kolesnikov, V. & Lee, W. Cao, J., Au, M. H., Lin, Z. & Yung, M. (eds) *Cryptographic key derivation from biometric inferences for remote authentication.* (eds Cao, J., Au, M. H., Lin, Z. & Yung, M.) *ASIACCS 21*, 629–643 (ACM Press, 2021).

[25] ISO/IEC. Information technology — biometric presentation attack detection — part 1: Framework (2023). ISO/IEC 30107-1:2023.

[26] Marcel, S., Nixon, M. S., Fierrez, J. & Evans, N. (eds) *Handbook of Biometric Anti-Spoofing: Presentation Attack Detection and Vulnerability Assessment* 3rd edn (Springer, 2023).

[27] Daugman, J. How iris recognition works (2004).

[28] of Standards, N. I., (NIST), T. & Dworkin, M. J. Sha-3 standard: Permutation-based hash and extendable-output functions (2015). URL https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=919061.

[29] Deng, J., Guo, J., Xue, N. & Zafeiriou, S. Arcface: Additive angular margin loss for deep face recognition (2019).

[30] Hoe, J. T. *et al.* One loss for all: deep hashing with a single cosine similarity based learning objective (2021).

[31] Shoshan, A., Bhonker, N., Kviatkovsky, I. & Medioni, G. Gan-control: Explicitly controllable gans (2021).

[32] Grother, P., Ngan, M. & Hanaoka, K. Face recognition vendor test (frvt) part 2: Identification (2025). URL https://pages.nist.gov/frvt/reports/1N/frvt_1N_report.pdf.

[33] Goldreich, O., Goldwasser, S. & Micali, S. How to construct random functions. *Journal of the ACM* **33**, 792–807 (1986).

[34] Muñoz, A., Ríos, R., Román, R. & López, J. A survey on the (in)security of trusted execution environments. *Computers & Security* **129**, 103180 (2023).

[35] Valiant, G. & Valiant, P. Fortnow, L. & Vadhan, S. P. (eds) *Estimating the unseen: an $n/\log(n)$-sample estimator for entropy and support size, shown optimal via new CLTs.* (eds Fortnow, L. & Vadhan, S. P.) *43rd ACM STOC*, 685–694 (ACM Press, 2011).

[36] Valiant, G. & Valiant, P. A clt and tight lower bounds for estimating entropy (2010).

[37] Hsiao, C.-Y., Lu, C.-J. & Reyzin, L. Naor, M. (ed.) *Conditional computational entropy, or toward separating pseudoentropy from compressibility.* (ed.Naor, M.) *EUROCRYPT 2007*, Vol. 4515 of *LNCS*, 169–186 (Springer, Berlin, Heidelberg, 2007).

[38] Håstad, J., Impagliazzo, R., Levin, L. A. & Luby, M. A pseudorandom generator from any one-way function (1999). URL https://doi.org/10.1137/S0097539793244708. https://doi.org/10.1137/S0097539793244708.

[39] Barak, B., Shaltiel, R. & Wigderson, A. Computational analogues of entropy (2003).

[40] Lynn, B., Prabhakaran, M. & Sahai, A. Cachin, C. & Camenisch, J. (eds) *Positive results and techniques for obfuscation.* (eds Cachin, C. & Camenisch, J.) *EUROCRYPT 2004*, Vol. 3027 of *LNCS*, 20–39 (Springer, Berlin, Heidelberg, 2004).

[41] Canetti, R. & Dakdouk, R. R. Smart, N. P. (ed.) *Obfuscating point functions with multibit output.* (ed.Smart, N. P.) *EUROCRYPT 2008*, Vol. 4965 of *LNCS*, 489–508 (Springer, Berlin, Heidelberg, 2008).

[42] Bellare, M. & Rogaway, P. Denning, D. E., Pyle, R., Ganesan, R., Sandhu, R. S. & Ashby, V. (eds) *Random oracles are practical: A paradigm for designing efficient protocols.* (eds Denning, D. E., Pyle, R., Ganesan, R., Sandhu, R. S. & Ashby, V.) *ACM CCS 93*, 62–73 (ACM Press, 1993).

# A Security

Here we present our formal security model for biometric authentication systems. We start by providing some preliminary definitions that we will use in our security model and proof.

## A.1 Model Preliminaries

We start with the standard notion of *min-entropy*, which is a measure of the unpredictability of a random variable. Intuitively, the best that an adversary can do to guess $A$ is to guess the most likely value of $A$, which has probability $\max_a \Pr[A = a]$.

**Definition 1** (Min-Entropy) The *min-entropy* of a random variable $A$ is defined as:

$$H_\infty(A) = -\log_2\left(\max_a \Pr[A = a]\right).$$

Min-Entropy is only defined for a random variable in isolation. To measure the min-entropy in the presence of other correlated random variables, i.e., information stored in a database,[21] introduced the following notion of *average min-entropy*. This notion intuitively defines how unpredictable a random variable remains, even given the correlated information, e.g., the database, and thus corresponds to security of the database.

**Definition 2** (Average Min-Entropy) The *average min-entropy* of a random variable $A$ conditioned on another random variable $B$ is defined as:

$$\tilde{H}_\infty(A \mid B) = -\log_2\left(\mathbb{E}_{b \leftarrow B}\left[\max_a \Pr\left[A = a \mid B = b\right]\right]\right) = -\log_2\left(\mathbb{E}_{b \leftarrow B}\left[2^{-H_\infty(A|B=b)}\right]\right).$$

See[21] for more details, including why it is important to take the log outside of the expectation, for cryptographic applications.

Average min-entropy is a purely information-theoretic notion of security, meaning that security holds even against adversaries with unbounded computational resources. However, in this paper we use cryptographic tools that are secure against (weaker, but more realistic) polynomial-time adversaries to enable our more performant system. Therefore, we use the computational version of average min-entropy, conditional HILL entropy, introduced in[37] Definition 3 (which in turn is based on notions from[38,39]).

First, we provide the standard notion of computational distance between distributions, commonly used in the cryptographic literature. Typically, we call some system *secure* if the computational distance between any information that the adversary can infer from the real system and any information that the adversary can obtain from an ideal system that is independent of the secret is *negligible* (i.e., super-polynomially small) in some given security parameter, $\lambda$.

**Definition 3** (Computational Distance) Let $X$ and $Y$ be two distributions. For a distinguisher $D$, we write the *computational distance* between $X$ and $Y$ as

$$\delta^D(X, Y) = |\Pr[D(X) = 1] - \Pr[D(Y) = 1]|.$$

We say that the distance $\delta^D(X, Y)$ is *negligible* in some security parameter $\lambda$, or $\delta^D(X, Y) = \mathsf{negl}(\lambda)$, if for every polynomial $p(\cdot)$, there exists $\lambda_0$ such that for every $\lambda \geq \lambda_0$, we have $\delta^D(X, Y) < 1/p(\lambda)$.

**Definition 4** (Conditional HILL Entropy) Let $(W, S)$ be a pair of random variables. $W$ has *HILL entropy at least $m$ conditioned on $S$*, denoted $H_{\mathsf{HILL}}(W \mid S) \geq m$ if there exists a joint distribution $(X, S)$ such that $\tilde{H}_\infty(X \mid S) \geq m$ and $\delta^D((W, S), (X, S)) = \mathsf{negl}(\lambda)$ for any probabilistic polynomial-time distinguisher $D$.

Conditional HILL entropy says that no adversary can distinguish the real joint distribution of the biometric data $W$ and the database $S$ from an ideal distribution in which a different distribution $X$ is swapped in for $W$ that has high average min-entropy given the database $S$, meaning that $X$ is unpredictable even given the database. Therefore, security of the biometric authentication system follows.

As a stepping stone to prove the full security of our biometric authentication system, we use a notion called *virtual black box obfuscation*[40] Definition 2. In essence, each hash $h$ we store in the database can be thought of as an obfuscation of a point function $P_v$ defined by $P_v(x) = 1$ if $x = v$ and 0 otherwise, where $v$ is the substring of the LSH output that we cryptographically hash to obtain $h$.

**Definition 5** (Virtual Blackbox Obfuscation in the Random Oracle Model) An oracle algorithm $\mathcal{O}$ is a *virtual blackbox obfuscator in the random oracle model* for a function family $\mathcal{F}$ if for every probabilistic polynomial-time adversary $\mathcal{A}$, there exists a probabilistic polynomial-time simulator $\mathcal{S}$ such that for every $F \in \mathcal{F}$, we have

$$|\Pr[\mathcal{A}^{\mathsf{RO}}(\mathcal{O}^{\mathsf{RO}}(F)) = 1] - \Pr[\mathcal{S}^F(1^\lambda) = 1]| \leq \mathsf{negl}(\lambda),$$

where $\mathsf{RO}$ is the random oracle.

In,[40] Lemma 6 it is indeed shown that $h \leftarrow \mathsf{CH}(v)$ which we store in our database is a virtual blackbox obfuscation of the point function $P_v$ in the random oracle model. At a high level, the simulator $\mathcal{S}$ with blackbox access to $P_v$ can simply sample a random value $h$ and run the adversary $\mathcal{A}$ on input $h$. Proofs in the random oracle model typically allow the simulator to answer the adversary's queries to the random oracle $\mathsf{RO}$. Thus, whenever $\mathcal{A}$ queries the random oracle $\mathsf{RO}$ on some input $x$, $\mathcal{S}$ can query $P_v$ on $x$; if $P_v(x) = 1$, then $\mathcal{S}$ can answer $\mathcal{A}$'s query with $h$, and otherwise $\mathcal{S}$ can answer with a random value.

**Lemma 1** (Obfuscation of Point Functions in the Random Oracle Model) Let $P_v : \{0,1\}^k \to \{0,1\}$ be a function defined by $P_v(x) = 1$ if $x = v$ and 0 otherwise. Define $\mathcal{P} = \{P_v : v \in \{0,1\}^k\}$. Then there exists a virtual blackbox obfuscator $\mathcal{O}$ for $\mathcal{P}$ in the random oracle model with security loss 0, which obfuscates given $P_v$ by storing $r = \mathsf{RO}(v)$ and on input $x \in \{0,1\}^k$, checks if $\mathsf{RO}(x) = r$; if so outputs 1, else 0.

Note that recent literature on Fuzzy Extractors often make use of *digital lockers*[41], which are related to obfuscations of point functions, but obfuscate the function $P_{v,s}$ defined by $P_{v,s}(x) = s$ if $x = v$ and $\perp$ otherwise.

### A.2 Our Formal Model

Finally, we present our formal model. We call a biometric authentication protocol $\mathsf{SBA} = (\mathsf{Bio\text{-}Setup}, \mathsf{Bio\text{-}Enroll}, \mathsf{Bio\text{-}Auth})$ a *Secure Biometric Authentication protocol with Cloud Database* if it satisfies the following security definition. Intuitively, the protocol is secure if for every enrolled identity, the biometric data of that identity still has high HILL entropy even given the public parameters and the entire database of enrolled users, meaning that the biometric data is computationally unpredictable even given the database.

**Definition 6** (Secure Biometric Authentication with Cloud Database) Consider any $N = \mathsf{poly}(\lambda)$, and any probabilistic polynomial-time adversary $\mathcal{A}$. Let $W_1, \ldots, W_N \leftarrow_{\$} \mathcal{V}$ be random variables representing the biometric samples of $N$ identities. Let $\mathsf{pp} \leftarrow_{\$} \mathsf{Bio\text{-}Setup}(1^\lambda)$ be the public parameters of the system. For $i \in [N]$, execute $\mathsf{pub}_i \leftarrow_{\$} \mathsf{Bio\text{-}Enroll}(\mathsf{pp}, W_i)$. Biometric Authentication protocol $\mathsf{SBA} = (\mathsf{Bio\text{-}Setup}, \mathsf{Bio\text{-}Enroll}, \mathsf{Bio\text{-}Auth})$ is *secure* if for every $i \in [N]$:

$$H_{\mathsf{HILL}}(W_i \mid (\mathsf{pp}, \mathsf{pub}_1, \ldots, \mathsf{pub}_N)) \geq \lambda.$$

*Remark 1* (Correctness) One could include a formal correctness notion for biometric authentication as well, however, since our system only provides empirical evidence of correctness, we omit it.

## B Formal Protocol Specification

In Algorithms 1-3 below, we formally present our entire biometric authentication protocol.

For notation, we use $n$ for the output length of the LSH; $m$ for the number of subsamples; $k$ for the length of the subsamples. We use the $\mathsf{LSH}$ to denote the used locality-sensitive hash and let $\mathsf{CH}$ to denote the cryptographic hash function, which is modeled as a random oracle[42] in our security proof below.

The setup algorithm presented in Algorithm 1 samples $m$ random subsets of $[n]$ of size $k$ according to some distribution $\mathcal{D}$. These subsets will correspond to the substrings of LSH output that we cryptographically hash during enrollment and authentication, as shown below.

The enrollment algorithm presented in Algorithm 2 takes as input a user identity $\mathsf{ID}$ and biometric data $x$, and outputs a set of hash values that are stored in the server's database and map to $\mathsf{ID}$. The algorithm first runs the data $x$ through a templatization model $\mathcal{M}$ to obtain a template $u$, then applies the LSH to obtain a bit string $v$, and finally hashes the subsamples of $v$ according to the subsets $\mu_i$ obtained during setup, to obtain hash values $h_i$ that are sent to the server for storage.

Finally, the authentication algorithm presented in Algorithm 3 takes as input biometric data $x$ and outputs a user identity $\mathsf{ID}$ if the authentication is successful, or

**Algorithm 1** Bio-Setup

**Parameters:** Distribution $\mathcal{D}$ over $[n]$ assigning weight $d_j$ to each bit $j \in [n]$

**for** $i \in [m]$ **do**
  Sample $\mu_i = \{j_1, \ldots, j_k\} \subset [n]$ by drawing $k$ elements without replacement from $\mathcal{D}$
**end for**
**return** $\mathsf{pp} \leftarrow (\mu_1, \ldots, \mu_m)$

**Algorithm 2** Bio-Enroll

**Parameters:** Templatization model $\mathcal{M}$, locality-sensitive hash $\mathsf{LSH}$

$u \leftarrow \mathcal{M}(x)$ ▷ Extract template from image
$v \leftarrow \mathsf{LSH}(u)$ ▷ Map template to bit string
**for** $i \in [m]$ **do** ▷ Hash each subsample
  $v_{\mu_i} \leftarrow v_{j_1} \| \ldots \| v_{j_k}$ where $\mu_i = \{j_1, \ldots, j_k\}$
  $h_i \leftarrow \mathsf{CH}(v_{\mu_i})$
**end for**
Send $(h_1, \ldots, h_m)$ to the server
Server inserts $(h_i, \mathsf{ID})$ into hash map $\mathsf{DB}$ for each $i \in [m]$

$\perp$ if it fails. The algorithm first runs the data $x$ through the same steps as enrollment to obtain hash values $h_i$, and then sends these hash values to the server, who looks them up in the database and returns the identity $\mathsf{ID}$ that has at least $\tau$ matches, where $\tau$ is a threshold parameter that can be tuned to achieve the desired trade-off between false positives and false negatives.

# C Security Proof

Finally, we prove that our biometric authentication system presented in the previous section is secure.

**Theorem 1** *Consider,* SBA *presented in Algorithms 1-3. Assume that for every $i \in [\eta]$, the distribution $\mathcal{V}_i$ of the substring of LSH output $V$ corresponding to subset $\mu_i$ sampled during* Bio-Setup *of Algorithm 1 has min-entropy at least $\lambda$, i.e., $H_\infty(\mathcal{V}_i) \geq \lambda$ for every $i \in [\eta]$. Then* SBA *is a secure biometric authentication protocol with external database.*

*Proof* Fix some $j \in [N]$. Let $V_i'$ be a random variable drawn from the distribution of substrings of LSH outputs corresponding to subset $\mu_i$ that is independent of $(\mathsf{pp}, \mathsf{pub}_1, \ldots, \mathsf{pub}_N)$ (and in particular, $\mathsf{pub}_j$). Then, we have $\tilde{H}_\infty(V_i' \mid (\mathsf{pp}, \mathsf{pub}_1, \ldots, \mathsf{pub}_N)) = H_\infty(V_i) \geq \lambda$.

It remains to show that $\delta((V_i, (\mathsf{pp}, \mathsf{pub}_1, \ldots, \mathsf{pub}_N)), (V_i', (\mathsf{pp}, \mathsf{pub}_1, \ldots, \mathsf{pub}_N))) = \mathsf{negl}(\lambda)$. First, by the security of the implicit obfuscation of point functions in our construction (Lemma 1), we have that for every distinguisher $D$,

$$|\Pr[D(V_i, (\mathsf{pp}, \mathsf{pub}_1, \ldots, \mathsf{pub}_N)) = 1]$$

**Algorithm 3** Bio-Auth($\mathsf{pp}, x$)

**Parameters:** Templatization model $\mathcal{M}$, locality-sensitive hash $\mathsf{LSH}$, threshold $\tau$

$u \leftarrow \mathcal{M}(x)$ ▷ Extract template from image
$v \leftarrow \mathsf{LSH}(u)$ ▷ Map template to bit string
**for** $i \in [m]$ **do** ▷ Hash each subsample
  $v_{\mu_i} \leftarrow v_{j_1} \| \ldots \| v_{j_k}$ where $\mu_i = \{j_1, \ldots, j_k\}$
  $h_i \leftarrow \mathsf{CH}(v_{\mu_i})$
**end for**
Send $(h_1, \ldots, h_m)$ to the server
Initialize dictionary $\mathsf{counts}[\cdot] \leftarrow 0$
**for** $i \in [m]$ **do**
  **if** $h_i \in \mathsf{DB}$ **then**
    $\mathsf{counts}[\mathsf{DB}[h_i]] \leftarrow \mathsf{counts}[\mathsf{DB}[h_i]] + 1$
  **end if**
**end for**
**for** $\mathsf{ID} \in \mathsf{counts}$ **do**
  **if** $\mathsf{counts}[\mathsf{ID}] \geq \tau$ **then**
    **return** $\mathsf{ID}$
  **end if**
**end for**
**return** $\perp$

$$- \Pr[\mathcal{S}^{P_{V_i}}(\mathsf{pp}, \mathsf{pub}_1, \ldots, \mathsf{pub}_{j-1}, \mathsf{pub}'_j, \mathsf{pub}_{j+1}, \ldots, \mathsf{pub}_N) = 1]| = 0, \tag{C1}$$

where $\mathsf{pub}'_j$ is the same as $\mathsf{pub}_j$, except it excludes the hash value corresponding to $v_i$.

Next, since $H_\infty(\mathcal{V}_i) \geq \lambda$, we have that

$$\Pr[\mathcal{S}^{P_{V_i}}(\mathsf{pp}, \mathsf{pub}_1, \ldots, \mathsf{pub}_{j-1}, \mathsf{pub}'_j, \mathsf{pub}_{j+1}, \ldots, \mathsf{pub}_N) = 1] - \Pr[D((V'_i, (\mathsf{pp}, \mathsf{pub}_1, \ldots, \mathsf{pub}_N))) = 1] = \mathsf{negl}(\lambda). \tag{C2}$$

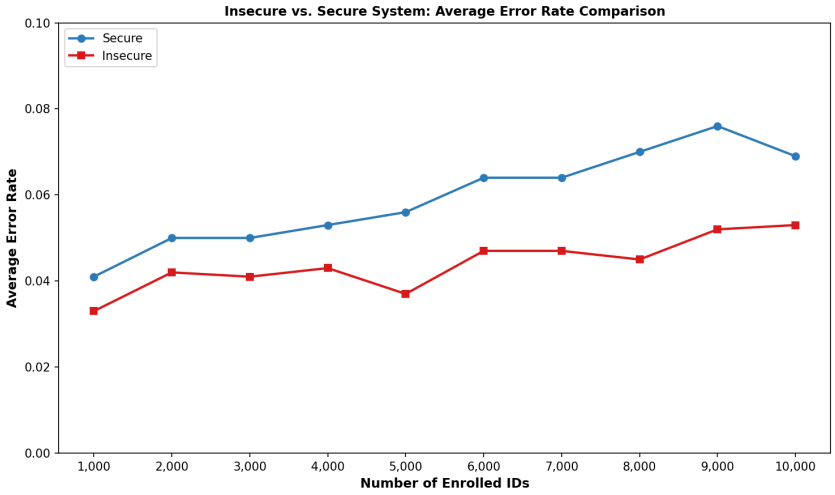
Indeed, $\mathcal{S}$ will never be able to guess $V_i$ and thus it as if all other information is independent of $V_i$, which is the case in the second term of Equation C2.

Adding up equations C1 and C2, we get negligible total computational distance. □

# D Extended Data

| Substring Length (bits) | Min | Max | Mean |
|---|---|---|---|
| 64 | 47.28 | 56.71 | 52.35 |
| 96 | 65.54 | 79.30 | 72.87 |
| 110 | 74.3 | 87.70 | 80.93 |

**Extended Data Table 1**: Statistics on min-entropy estimates across substring lengths.




**Extended Data Figure 1**: Error rates for databases of size 1,000-10,000 for both the secure and insecure baseline systems. False negatives are calculated over first 1000 enrolled identities. False positives are calculated over 1000 non-enrolled identities. We average false negative and false positive rates for clarity. Runs of our secure system are averaged over 5 trials.